Kịch bản trình bày báo cáo tuần

Nhóm 10 - Môn 608819 - Phát triển ứng dụng phía máy chủ nâng cao (lớp 68CS1&2) Thời lượng: khoảng 18 phút nói + 2-3 phút demo + hỏi đáp. Bám theo dàn ý 14 slide ở tài liệu 00.

Cách dùng

- Câu in nghiêng là lời nói gợi ý. Đọc thoáng qua rồi nói lại bằng lời của mình, đừng đọc nguyên văn.
- Mục "Chỉ vào" là sơ đồ hoặc bảng cần mở trên màn hình khi nói.
- Cột "Người" để nhóm tự điền (chia đều 5 người, mỗi người 2-3 slide liền nhau để đỡ đổi người liên tục).
- Quy tắc nói thật: cái gì chạy được thì nói "Đã có", cái gì mới thiết kế thì nói "Dự kiến". Không nói "hệ thống đã làm được" cho phần chưa có code.

Bảng thời gian

Slide	Nội dung	Thời gian	Người
1	Trang bìa	0:30	
2	Bài toán và phạm vi	1:30	
3	Tác nhân và yêu cầu chức năng	1:30	
4	Yêu cầu phi chức năng	1:00	
5	Sơ đồ phân rã chức năng	1:30	
6	Kiến trúc tổng thể	1:30	
7	Modular Monolith và Clean Architecture	2:00	
8	Integration event và luồng ghi danh	2:00	
9	Upload file bằng presigned URL	1:00	
10	Nguyên tắc cơ sở dữ liệu	1:30	
11	ERD	1:30	
12	Công nghệ và triển khai	1:00	
13	Tiến độ và demo	3:00	
14	Kế hoạch tuần tới và câu hỏi	1:00	
	Tổng	khoảng 21:00	

Nếu bị giới hạn 15 phút: nói lướt slide 4, 9, 12 (mỗi slide 30 giây) và giữ nguyên demo.

Slide 1 - Trang bìa (0:30)

Chỉ vào: tên đề tài, nhóm 10, môn 608819, lớp 68CS1&2, 5 thành viên.

"Em xin chào thầy. Nhóm 10 xin báo cáo đề tài Hệ thống quản lý trung tâm Tiếng Anh, backend .NET theo kiến trúc Modular Monolith. Hôm nay nhóm trình bày bốn tài liệu thầy yêu cầu: yêu cầu chức năng và phi chức năng, sơ đồ phân rã chức năng, thiết kế hệ thống, và thiết kế cơ sở dữ liệu kèm ERD."

Slide 2 - Bài toán và phạm vi (1:30)

Chỉ vào: 01 mục 1 và 6.

"Bài toán là quản lý một trung tâm Tiếng Anh: tạo lớp, xếp lịch, ghi danh học viên, điểm danh, giao bài và chấm bài, lập hóa đơn và gửi thông báo. Nhóm tham khảo luồng nghiệp vụ của một hệ LMS trung tâm Tiếng Anh, nhưng tự thiết kế giao diện và dữ liệu, không dùng logo hay nội dung của đơn vị nào."

"Phạm vi gồm 33 user story, chia ba mức: Must 20, Should 9, Could 4. Phần Could, như Xu hay bảng tin, chỉ làm nếu còn thời gian. Nhóm đã chủ động cắt các tính năng gamification, mini game, khuyến mãi, cửa hàng, thanh toán online và nhiều chi nhánh, vì không phải nghiệp vụ cốt lõi và làm tăng độ phức tạp mà không giúp đạt mục tiêu môn học."

Chuyển slide: *"Tiếp theo là ai dùng hệ thống và hệ thống phải làm gì."*

Slide 3 - Tác nhân và yêu cầu chức năng (1:30)

Chỉ vào: 01 mục 2 và 3, bảng FR-01..FR-33.

"Hệ thống có ba vai trò: Học viên, Giáo viên và Staff, trong đó Staff gộp Giáo vụ và Admin. Ngoài ra có tác nhân Hệ thống cho các việc tự động như tạo hóa đơn. Ảnh tham khảo chỉ có giao diện học viên, nên hai vai trò Giáo viên và Staff là nhóm suy ra từ dữ liệu học viên nhìn thấy."

"Yêu cầu chức năng đánh mã FR-01 đến FR-33, mỗi mã ứng với một user story. Có hai quy tắc nghiệp vụ nhóm muốn nhấn mạnh: điểm danh có 6 trạng thái, và hóa đơn được tạo tự động theo cơ chế idempotent, nghĩa là một lần ghi danh chỉ ra đúng một hóa đơn dù sự kiện bị xử lý lặp."

Slide 4 - Yêu cầu phi chức năng (1:00)

Chỉ vào: 01 mục 4, bảng NFR-01..NFR-18.

"Có 18 yêu cầu phi chức năng, chia theo hiệu năng, bảo mật, dễ sử dụng, bảo trì, kiểm thử và triển khai. Mỗi yêu cầu có cách đo kèm theo. Các con số như p95 dưới 500 ms hay 100 người dùng đồng thời là mục tiêu thiết kế, nhóm chưa đo thực tế vì chưa có nghiệp vụ để đo."

Slide 5 - Sơ đồ phân rã chức năng (1:30)

Chỉ vào: 02 mục 2, sơ đồ cấp 0-1 và ma trận chức năng - tác nhân.

"Hệ thống phân rã thành 8 nhánh chức năng F1 đến F8, mỗi nhánh có các chức năng con đánh mã F-x.y. Em xin chỉ ví dụ một nhánh để thầy thấy cách phân rã, các nhánh còn lại cùng cấu trúc. Ma trận chức năng - tác nhân cho biết vai trò nào được dùng chức năng nào, và là căn cứ để làm phân quyền ở bước code."

Gợi ý: chọn nhánh điểm danh hoặc bài tập để minh họa, vì sau đó sẽ gặp lại ở slide 8.

Slide 6 - Kiến trúc tổng thể (1:30)

Chỉ vào: 03 mục 2.

"Luồng tổng thể: trình duyệt đi qua Cloudflare tới VPS, Nginx làm reverse proxy, phía sau là API .NET, PostgreSQL và MinIO hoặc R2 để lưu file, Jenkins lo CI/CD. Em nói rõ: phần chạy được hiện nay là API .NET. Docker Compose, Jenkinsfile, frontend React, MinIO là Dự kiến, sẽ làm từ sprint tuần 2."

Slide 7 - Modular Monolith và Clean Architecture (2:00)

Chỉ vào: 03 mục 3 và 4, ma trận tham chiếu giữa các module.

"Nhóm chọn Modular Monolith thay vì microservices vì nhóm 5 người, một VPS, chi phí vận hành microservices cao. Hệ thống có 5 module: Identity, Academic, Enrollment, Finance và MediaNotification."

"Mỗi module có 5 project theo Clean Architecture: Domain, Contracts, Application, Infrastructure, Api. Mỗi module có DbContext và schema riêng. Điểm quan trọng nhất: module không được tham chiếu trực tiếp module khác, chỉ giao tiếp qua integration event hoặc contract công khai tối thiểu. Quy tắc này được bảo vệ bằng architecture test dùng NetArchTest, và cấu hình build chặn tham chiếu bắc cầu. Nếu ai thêm tham chiếu chéo, test sẽ đỏ."

Phần này là Đã có, có thể nói chắc.

Slide 8 - Integration event và luồng ghi danh (2:00)

Chỉ vào: 03 mục 6 và 7.2, bảng 10 event và sơ đồ tuần tự ghi danh.

"Các module nói chuyện bằng integration event qua Event Bus in-memory. Có 10 event được thiết kế, trong đó 7 đã có trong Contracts và 3 là Dự kiến: AssignmentMissed, InvoicePaid, WarningIssued."

"Ví dụ luồng ghi danh: Staff ghi danh học viên vào lớp, module Enrollment phát StudentEnrolled. Module Finance nhận và tự tạo hóa đơn, rồi phát InvoiceCreated, module MediaNotification nhận và gửi thông báo cho học viên. Ba module không biết nhau, chỉ biết event. Handler hóa đơn mẫu cho StudentEnrolled đã có trong code."

Nói thật về giới hạn nếu thầy hỏi: Event Bus hiện đồng bộ và in-memory, handler lỗi chỉ ghi log, không retry. Hướng khắc phục dự kiến là Outbox cộng retry (tài liệu 03 mục 11).

Slide 9 - Upload file bằng presigned URL (1:00)

Chỉ vào: 03 mục 7.3.

"Với file như bài nộp hay ảnh đại diện, client xin API một presigned URL rồi upload thẳng lên MinIO hoặc R2, không đi qua API. Sau đó client chỉ gửi lại fileKey cho API. API kiểm tra file thực sự đã lên, đúng định dạng và không quá 20 MB bằng một lệnh HEAD. Cách này giữ API nhẹ và không lưu file trên server."

Slide 10 - Nguyên tắc cơ sở dữ liệu (1:30)

Chỉ vào: 04 mục 1 và 2, sơ đồ tổng quan tham chiếu logic.

"Một database PostgreSQL, 5 schema, mỗi schema do đúng một DbContext quản lý. Quy tắc cứng: không join, không view, không khóa ngoại giữa các schema. Tham chiếu sang module khác chỉ là ID logic kiểu uuid, đồng bộ bằng event."

"Khi một module cần dữ liệu của module khác, nó giữ một bản sao chỉ đọc gọi là read model, cập nhật bằng event. Ví dụ Enrollment giữ class_snapshots để biết học phí và sĩ số của lớp. Cái giá là nhất quán cuối cùng: dữ liệu có thể chậm một nhịp, nhóm chấp nhận điều đó."

Slide 11 - ERD (1:30)

Chỉ vào: 04 mục 3, ERD schema `academic` rồi lướt 4 schema còn lại.

"Đây là ERD của schema academic, schema lớn nhất: khóa học, lớp, buổi học, điểm danh, bài tập và bài nộp. Unit và Lesson là cột của buổi học. Điểm chấm theo thang 0 đến 10. Bốn schema còn lại nhỏ hơn: identity có người dùng, vai trò và refresh token; enrollment có ghi danh, cảnh báo và bản chụp điểm danh; finance chỉ có hóa đơn; media_notification có file, thông báo và phản hồi."

"Toàn bộ bảng này là thiết kế, chưa có migration. Hiện mới có 5 DbContext rỗng với schema riêng."

Slide 12 - Công nghệ và triển khai (1:00)

Chỉ vào: 03 mục 9, 10, 12.

"Công nghệ: .NET 10, EF Core 10, Npgsql, PostgreSQL, frontend React với Vite. Triển khai: Docker Compose multi-stage để chạy cả hệ thống bằng một lệnh, Jenkins cho CI/CD, VPS và Cloudflare. Các phần triển khai đều là Dự kiến. Về quản lý công việc, nhóm dự kiến dùng Jira từ giai đoạn code, khi có sprint thật."

Slide 13 - Tiến độ và demo (3:00)

Đây là phần duy nhất cho thấy sản phẩm chạy thật. Chuẩn bị sẵn terminal tại `d:\english-center` trước khi vào phòng. Chi tiết kịch bản demo ở tài liệu 00 mục 4.

"Tuần 1 nhóm hoàn thành khung solution, chưa làm nghiệp vụ. Em xin demo ba việc."

1. Chạy host bằng `dotnet run --project src/Host/EnglishCenter.Api`. Mở Swagger tại `http://localhost:5080/swagger`. *"Đây là API của 5 module trên một host."*
2. Gọi lần lượt các endpoint `/api/identity/ping`, `/api/academic/ping`, `/api/enrollment/ping`, `/api/finance/ping`, `/api/media-notification/ping`, mỗi cái trả 200 với tên module và schema. Gọi thêm `/health` và nói kết quả `Healthy`.
3. Dừng host, chạy `dotnet test EnglishCenter.sln`. *"79 test, gồm architecture test chặn tham chiếu chéo, test Event Bus và integration test, đều xanh."*

Phương án dự phòng nếu lỗi máy hoặc mạng: mở ảnh chụp sẵn của Swagger và kết quả `dotnet test`. Nên chụp trước và để cùng thư mục với slide.

Lưu ý: không hứa demo đăng nhập, ghi danh hay hóa đơn, vì các luồng đó chưa có code.

Slide 14 - Kế hoạch tuần tới và câu hỏi (1:00)

Chỉ vào: backlog mục 7.

"Kế hoạch tuần 2: Docker Compose, đăng nhập, đăng xuất, đổi mật khẩu, tạo tài khoản, phân quyền, tạo lớp và dịch vụ lưu file. Tuần 3: lịch, danh sách lớp, ghi danh, hóa đơn. Tuần 4: điểm danh, bài tập, nộp bài, chấm điểm, thông báo."

"Nhóm xin thầy góp ý ba điểm: phạm vi 33 user story có hợp lý với thời gian không; cách chia schema và read model có ổn không; và nhóm nên ưu tiên phần nào nếu thiếu thời gian. Em xin cảm ơn thầy."

Hỏi đáp

Bảng câu hỏi thầy có thể hỏi và gợi ý trả lời nằm ở tài liệu 00 mục 5. Cả nhóm nên đọc trước. Bốn câu dễ bị hỏi nhất:

Câu hỏi	Ý chính trả lời	Người trả lời
Vì sao không dùng microservices?	Nhóm nhỏ, một VPS; Modular Monolith giữ ranh giới bằng architecture test, tách service sau được vì chỉ giao tiếp qua event	
Thống kê khi không join chéo schema thì sao?	Mỗi module có API số liệu riêng, frontend tổng hợp; không truy vấn chéo schema	
Mất event thì sao?	Hiện in-memory, handler lỗi chỉ ghi log, chưa retry; hướng khắc phục là Outbox cộng retry	
Vì sao trùng dữ liệu giữa module?	Để không join chéo schema; bản sao chỉ cập nhật bằng event, chấp nhận nhất quán cuối cùng	

Nguyên tắc khi trả lời: nếu thầy hỏi về thứ chưa làm, nói thẳng "phần này nhóm đang để Dự kiến, thiết kế ở tài liệu X mục Y", không nói quanh.

Danh sách chuẩn bị trước buổi báo cáo

- Điền tên người trình bày vào bảng thời gian và bảng hỏi đáp.
- Chạy thử trọn vẹn phần demo một lần trên máy sẽ dùng, bấm giờ.
- Chụp sẵn ảnh Swagger, `/health` và kết quả `dotnet test` làm phương án dự phòng.
- Mở sẵn các sơ đồ Mermaid (trong VS Code Markdown Preview hoặc trên GitHub) để chỉ vào khi nói.
- Cả nhóm đọc qua bảng câu hỏi ở tài liệu 00 mục 5.